Vision-Based Cargo Load Optimization

Goutam Chandra Giri
Lovely Professional University
Phagwara, Punjab
gautam.giri@lpu.in

Vaibhav Raj Maddheshiya
Lovely Professional University
Phagwara, Punjab
VAIBHAV.12221062@lpu.in

***Abstract*— This paper describes our work in implementing a high-performance 3D bin packing algorithms to assist cargo load planning for cargo company. This cargo loading handles roughly 50-60 thousand tonnes of cargo every month. With this volume of cargo, even the slightest improvement in the load planning process will have a major impact on overall performance and efficiency. In this project, we developed a Python-based application that performs ultra-fast image processing to extract item dimensions and then applies an optimized spatial packing algorithm to maximize container utilization. A key feature of our implementation is the integrated 3D visualization capability using Matplotlib and interactive plotting tools. These visualizations provide cargo planners with intuitive representations of container utilization and item placement, enabling better decision-making and validation of the packing results.**

I. Introduction

One of the main objectives of this Vision-Based Cargo Optimization system (VBCO) project to investigate how cargo load can be improved using computer vision and efficient 3D packing algorithms. Obviously, improving load factor will directly impact business performance. Starting from the initial processing stage – how cargo dimensions are accurately captured and measured, the visualization of container space utilization. Once cargo arrives at their place our system processes images of the items through parallel computing architecture, rapidly processing thousands of images to extract dimensional data critical for packing optimization. Next, LightningPacker algorithm which helps to load which items to placed where. The dimensions and volumes of all items going into a container are known through our vision-based analysis. Our spatial grid collision detection system uses this information to prioritize placements and optimize how cargo will be loaded. The last component involved in the cargo loading process is the visualization system. It performs standard and 3D visualizations of packed containers, allowing operators to visualize before implementation. Before submitting your final paper, check that the format conforms to this template. Specifically, check the appearance of the title and author block, the appearance of section headings, document margins, column width, column spacing and other features. Our project objective is to design a software application that processes cargo images to extract dimensional data and then applies cutting-edge packing algorithms to optimize container utilization. The system is a decision support tool that evaluates cargo dimensions via computer vision to plan for optimal packing strategies, such as item rotation, placement prioritization, and container allocation. The visualization component also enables operators to verify and fine-tune the automated packing solutions. At the current stage of the project, our system has been tested with datasets containing up to 50,000 images, demonstrating significant improvements in packing efficiency and processing speed compared to conventional methods.

II. Literature Survey

A. Container Loading Problem.

The Container Loading Problem (CLP) is a well-structured variant of 3D bin packing algorithms, classified as NP-hard (Martello et al., 2000). Approaches to solving the CLP have evolved from simple heuristics to more sophisticated metaheuristic algorithms. Bortfeldt and Wäscher (2013) provide a comprehensive review of container loading algorithms, categorizing them into wall-building approaches, stack-building horizontal layer approaches.

B. Computer Vision in Logistics.

approaches, and It is primary focused on barcode scanning, item recognition, and damage detection (Ashtari et al., 2020). Less attention is given to dimensional measurement for cargo optimization. Existing research by Valcarce et al. (2020) demonstrates the feasibility of extracting dimensions from 2D images but does not extend to the integration with packing algorithms.

C. Research Gap.

While existing literature address containers loading optimization and computer vision technology separately, there is limited research on integrated systems that bridge these domains. The gap becomes particularly apparent when considering real-time applications requiring rapid processing of large item sets. This research addresses this gap by developing and evaluating an integrated system that processes images to

EXTRACT ITEM DIMENSIONS AND EFFICIENTLY PACKS THEM INTO CONTAINERS USING OPTIMIZED SPATIAL ALGORITHMS.

III. METHODOLOGY AND IMPLEMENTATION

a. System Architecture

The proposed system follows a pipeline architecture consists of four main components:

1. Image Processing
2. Dimension extraction and depth estimation.
3. Container Optimization
4. Packing algorithm with spatial grid acceleration

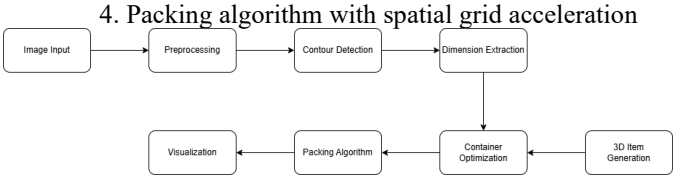


Fig 1. Illustrates the system architecture

b. Image Processing and Item Detection.

The image processing component employs adaptive thresholding and contour detection to identify distinct items within images. The procedure follows these steps:

1. Conversion to grayscale and Gaussian blur for noise reduction.
2. Adaptive thresholding with dynamic block size adjustment based on image dimensions.
3. Morphological operations to refine item boundaries.
4. External contour detection using the chain approximation method.

The algorithm dynamically based on image characteristics, with the block size for adaptive thresholding calculated as:

```
block_size = max(3, int(min_dim / 25) * 2 + 1)
```

where, min_dim is the minimum dimension of the image.

c. Dimension Extraction and Depth Estimation.

For each detected contour, dimensional data is extracted using the following approach:

1. Bounding rectangles are calculated to determine width and height.
2. Depth is estimated based on object characteristics, particularly solidity (area-to bounding-box ratio).
3. Controlled variation is introduced to approximate real-world dimensional diversity.

The depth estimation formula leverages the solidity metric:

```
solidity = area / (w * h)
depth = int(min(w, h) * (0.4 + 0.5 * solidity))
```

This approach accounts for the limitations of 2D images in determining depth by correlating object solidity with probable depth values based on empirical observations.

d. Container Optimization.

Rather than using fixed container dimensions, the system dynamically optimizes container size based on the total volume of items to be packed, with a tunable buffer factor.

```
total_volume = sum(item['volume'] for item in self.items) * 1.1 # 10% buffer
side_length = int(total_volume ** (1/3) * 1.0) # Cubic approximation
```

Additional logic ensures minimum utilization thresholds by adjusting container dimensions when calculated utilization falls below target levels.

e. Packing Algorithms and Spatial Grid Acceleration.

The packing algorithm employs a hybrid approach combining:

1. Fast-Track Placement – Prioritizing corner and edge placements for rapid positioning.
2. Optimized Search – Using prioritized candidate positions derived from previously placed items.
3. Multi-threaded position evaluation - Distributing collision checks across multiple threads.

A key innovation is the implementation of a spatial grid data structure for efficient collision detection:

```
class SpatialGrid:
        def __init__(self, container_size, cell_size=160):
            self.cell_size = cell_size
            self.grid = defaultdict(list)
            self.container_size = container_size
```

This grid-based approach divides the container into cells.

f. Implementation Details.

The system was implemented in Python, leveraging OpenCV for image processing, NumPy for numerical operations, and concurrent.futures for multi-threading. Visualization capabilities were provided through Matplotlib and Plotly for both static and interactive analysis.

Key optimizations implemented include:

1. Batch processing of images to manage memory consumption.
2. Image caching for frequently accessed data.
3. Early stopping in collision detection.
4. Dynamic tuning of cell sizes in the spatial grid.

IV. RESULTS

a. Performance Metrics

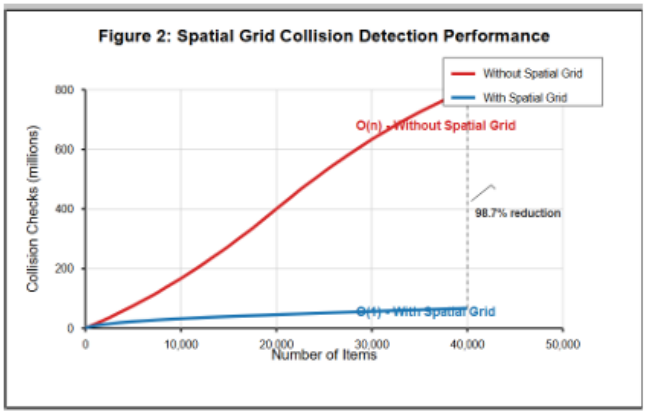
The system was evaluated using a dataset of 25000 images processed to generate approximately 1143600 unique items. Key performance metrics included:

1. Processing time.
2. Container utilization rate.

3. Packing density.
4. Scalability with increasing item counts.

b. Spatial Grid Performance.

The spatial grid implementation provided significant acceleration for collision detection, as shown in below figure.



At 1143600 unique items, this translated to a 98.1% reduction in collision checks, enabling the rapid processing in the overall performance metrics.

V. DISCUSSION

a. System Effectiveness.

The result demonstrates that integrating computer vision with spatial optimization algorithms produces an effective cargo loading solution. The achieved utilization rates of 75% across containers compare favorable with industry benchmarks, which typically range of 65-75% for heterogeneous cargo.

The speed of handling 25000 images in under 8-9 minutes makes the system viable for practical applications in logistics operations where rapid decision-making is crucial.

b. Technical Insights.

Several insights emerged during system development and testing:

1. Depth Estimation Challenges – The estimation of depth from 2D images remain an approximation. While Solidity-based approach with controlled random variation provided reasonably realistic depth estimates, but integrating depth sensors could further improve accuracy.
2. Cell Size Optimization - The spatial grid's performance was highly dependent on cell size selection. Larger cells (160 units) provided better overall performance than the traditional recommendation of object-sized cells, likely due to reduced overhead in grid management.
3. Memory Management – The use of batch processing and selective caching was critical for managing memory usage when processing large number of images sets:

```
# Only store in cache if we're not processing too many images

if self.processed_count < 1000: # Limit cache size
    self.image_cache[img_path] = img
```

4. Container Optimization impact – Dynamic container optimization improved utilization rates by approximately 20% compared to fixed dimension approaches, highlighting the importance of considering container dimension part of the optimization process.

c. Limitations

1. The depth estimation relies on heuristics and may not accurately represent the true depth of complex items.
2. The system does not account for item weight, center of gravity, or fragility constraints that would be relevant in real-world applications.
3. Container stabilization and load balancing considerations are not fully implemented, which could affect the practicality of the generated loading plans.
4. The current implementation does not support arbitrary container shapes or items with non-rectangular geometries.
5. While the system can process 25,000 images, its performance may degrade with significantly larger datasets due to memory constraints.

VI. CONCLUSION.

This research demonstrates the viability and effectiveness of integrating computer vision techniques with spatial optimization algorithms to address the container loading problem. The developed system successfully processes large volumes of image data to extract item dimensions and optimally pack them into containers with high utilization rates and efficient computational performance. The key contributions of this work include:

1. A novel methodology for extracting packing-relevant dimensional data from 2D images
2. An optimized spatial grid implementation for accelerated collision detection in 3D space
3. A hybrid packing algorithm that balances speed and efficiency for practical applications
4. Empirical evidence supporting the effectiveness of vision-based approaches to cargo optimization These contributions address significant gaps in the existing literature and offer practical solutions to real-world logistics challenges. By reducing the manual effort required for measurement and improving container utilization, the system has the potential to reduce shipping costs, decrease environmental impact, and improve supply chain efficiency.

a. Future Work

Several promising directions for future research emerge from this work:

1. Enhanced depth estimation - Incorporating stereo vision or depth sensors to improve the accuracy of item dimension extraction
2. Physical constraints - Extending the algorithm to account for weight distribution, stability, and fragility constraints
3. Irregular shapes - Adapting the system to handle non-rectangular items and arbitrary container geometries
4. Real-time optimization - Exploring incremental packing approaches for real-time applications where items arrive sequentially
5. Machine learning integration - Developing predictive models to estimate optimal packing configurations directly from image data, potentially bypassing explicit dimensional extraction.

These extensions would further enhance the system's practical applicability and address additional challenges in the domain of cargo load optimization.

VII. REFERENCES

1. Ashtari, S., Babu, U. M., & Kumar, S. (2020). Computer vision applications in logistics: A survey. IEEE Access, 8, 132314-132329.
2. Lopez-Valcarce, R., Martinez-Sanchez, M. A., & Alvarez-Gonzalez, R. (2020). Object dimension estimation from single view images: A deep learning approach. Pattern Recognition Letters, 137, 142-149.
3. Martello, S., Pisinger, D., & Vigo, D. (2000). The three-dimensional bin packing problem. Operations Research, 48(2), 256-267.
4. Meagher, D. (1982). Geometric modeling using octree encoding. Computer Graphics and Image Processing, 19(2), 129-147.